KristaTech (KRISTA) Technical Whitepaper

A Quorum-Resilient Cooperative Hybrid Consensus Blockchain with Proof-of-BLS (PoBLS) Proposer Selection and JSON-Compiled Declarative Smart Contracts (MESCAL)

Abstract
This paper introduces the KristaTech (KRISTA) blockchain protocol, a decentralized platform designed to address consensus centralization, leader-targeted Denial-of-Service (DoS) attacks, and the architectural complexities of traditional virtual machines. KristaTech implements a two-tier consensus mechanism that decouples validator set selection from block proposal. The selection layer, **ADAM (A Decentralized Approach Model)**, utilizes a Verifiable Random Function (VRF) rolling seed to deterministically elect a set of N validators and 1 coordinator per block height. The proposal layer, **Proof of BLS (PoBLS)**, holds an ephemeral cryptographic lottery via XOR distance calculation to dynamically select the block producer from the elected validator pool, securing the network against pre-computation and proposer-targeted DoS attacks. To maintain chain liveness under network partitions or offline validators, we introduce a quorum-resilient responding mechanism utilizing empty vector placeholders (`std::vector<unsigned char>()`) verified against a dynamic threshold. Smart contracts are executed using **MESCAL (Minimalistically Envisioned Smart Contract Assembling Language)**, a simple, declarative, JSON-based specification that compiles directly into stack-based `CScript` bytecode, eliminating state-based vulnerabilities and gas calculation overhead. Finally, we propose a sustainable **210 Million KRISTA** emission model governed by a **1.9% quarterly (~90 days) decay** and the **Model D Cooperative Split**, which fairly distributes block rewards among passive masternodes (50%), active LLMQ quorum members (10%), the block producer (15%), and validator participants (25%), alongside institutional developer and faucet funding allocations.

Keywords: Consensus Protocols, Blockchain Security, Cryptographic Lottery, Declarative Smart Contracts, Tokenomics.

---

1. Introduction and Background

Decentralized consensus protocols are fundamentally tasked with solving the Byzantine Generals Problem in an adversarial, open-network environment. Traditional consensus designs, primarily Proof of Work (PoW) and Proof of Stake (PoS), have successfully demonstrated network coordination but suffer from critical structural weaknesses:

1. **Consensus Centralization**: In PoW, the economies of scale lead to the concentration of hashing power in a few industrial mining pools. In PoS, wealth concentration (“rich-get-richer” dynamics) permits dominant coin holders to govern block production.
2. **Leader Targetability**: In protocols where the next block proposer is pre-elected or

predictable, attackers can launch targeted Denial-of-Service (DoS) attacks or bribe the proposer to censor transactions, compromising network liveness.

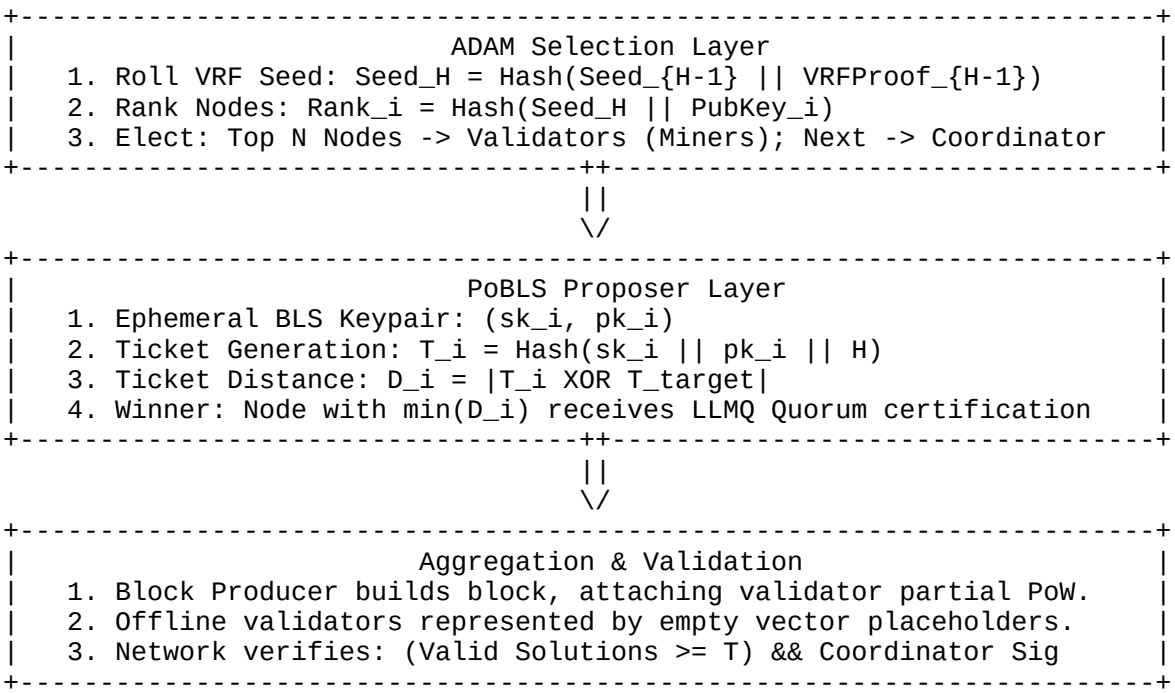
3. **Block Grinding Attacks**: If block headers or transactions can be manipulated to influence the entropy source of future leader elections, adversarial nodes can grind blocks to artificially increase their selection probability.
4. **Smart Contract Simplicity**: Traditional virtual machines introduce significant complexity in state execution, causing vulnerabilities and unpredictable transaction execution. A simpler, predictable contract language is required to guarantee state transitions.

KristaTech addresses these challenges through a dual-consensus cooperative architecture. By splitting consensus into a **deterministic selection layer** (ADAM) and an **unpredictable proposing layer** (PoBLS), KristaTech achieves high throughput, DDoS resilience, and robust Sybil resistance. Contract execution is restricted to the deterministic and simple **MESCAL** declarative language, maintaining blockchain state security. The ecosystem is supported by a deflationary, long-lifecycle tokenomics model designed for sustainable network maintenance over decades.

---

2. Consensus Architecture

The core of KristaTech is a cooperative consensus process. The block lifecycle is split into an **Election Phase**, a **Solving Phase**, and an **Aggregation Phase**.



2.1. A Decentralized Approach Model (ADAM)

ADAM serves as the identity and selection layer. Rather than allowing all nodes to compete freely, which introduces Sybil vulnerabilities, ADAM selects a restricted pool of N Validators (Miners) and 1 Coordinator per block height.

2.1.1. VRF Rolling Seeds

To prevent block grinding, the selection entropy is generated via a Verifiable Random Function (VRF). At block height H, the selection seed is defined as:

$$\text{Seed}_H = \text{Hash}\left(\text{Seed}_{H-1} \| \text{VRFProof}_{H-1}\right)$$

Where the VRF proof (VRFProof) at block height $H-1$ is a deterministic signature generated by the Coordinator of block $H-1$ using a hybrid BLS12-381 + ECDSA fallback signature mechanism (`SignBLSWithECDSAFallback`) on the seed of block $H-2$. Because the resulting BLS signature (authorized by an ECDSA signature of the BLS public key) is strictly deterministic, the Coordinator cannot manipulate the signature value to skew the election of block $H+1$. The rolling seed is stored directly in the block header, making the history of selection entropy immutable and publicly verifiable.

2.1.2. Node Selection & Ranking

Given the active node pool P (the set of registered Masternodes), the system calculates a unique rank for each node i:

$$\text{Rank}_i = \text{Hash}\left(\text{Seed}_H \| \text{PubKey}_i\right)$$

The node pool is sorted in ascending order of their Rank_i values. The first N nodes are elected as **Miners (Validators)**, and the $(N+1)$-th node is elected as the **Coordinator**. On Mainnet and Testnet, the selection pool is dynamically constructed from active Masternodes and active registered miners (via Coin-Lock or PoW-Lock). To ensure nodes can accumulate the necessary 4,200 KRISTA collateral for masternode setup during the bootstrap phase, Coin-Lock registration is disabled below block height 2,200 (Model D activation) on Mainnet, forcing nodes to use PoW-Lock (locking only 0.0001 KRISTA) and keeping block rewards liquid. On Regtest, the pool automatically includes 15 deterministic bootstrap public keys to facilitate automated testing.

2.1.3. Mode Dynamics & Spork-Control

To facilitate bootstrapping, ADAM operates in two modes: * **Fallback Mode (Version 11)**: Operates during the early bootstrap phase of the network. The validator pool is selected from the registered miner pool (since the active masternode count is below the quorum threshold). The miners count $N \in [11, 14]$ is dynamic, and the consensus threshold T is governed by

`GetAdamThreshold(nHeight)` (5 valid solutions during the bootstrap phase for `nHeight < nAdamThresholdBootstrapLimit`, i.e., below block 50,000 on Mainnet / 100,000 on Testnet / 1,000 on Regtest, transitioning thereafter to the consensus parameter `nAdamThreshold` of 7 valid solutions on Mainnet/Regtest and 3 on Testnet). * **Standard Mode (Version 12)**: Enforces full cooperative consensus once a sufficient number of active Masternodes are online. The miner count N is set to `nAdamMinersCount` (11), and the consensus threshold T is governed by `GetAdamThreshold(nHeight)` (5 solutions during bootstrap below block 50,000 on Mainnet / 100,000 on Testnet / 1,000 on Regtest, and 7 on Mainnet/Regtest or 3 on Testnet thereafter). The Coordinator is elected dynamically from the active Masternode list, while the miners are elected from the registered miner pool. * **Activation**: The transition is governed by `SPORK_21_ADAM_STANDARD_MODE` (Spork ID `10020`). If active, the protocol enforces Version 12 block validation.

2.1.4. Bootstrap Security and Starting Difficulty

To ensure the network can bootstrap securely from genesis (block 1) when mining is public and multiple nodes mine concurrently, the starting PoW difficulty limit (`powLimit`) is hardened to `~UINT256_ZERO >> 20` (equivalent to the genesis block's `nBits` of `0x1e0ffff0`). This prevents blocks 1–199 from being mined instantly in microseconds, giving nodes sufficient time to establish P2P connections and propagate blocks. This eliminates the risk of fork splits (where nodes disagree on the block 199 hash and fail to form quorums at block 200).

2.2. Proof of BLS (PoBLS) Consensus

Once the validator set is elected by ADAM, the block proposer is chosen using the **Proof of BLS (PoBLS)** lottery. This prevents proposer targetability, as the winning block producer is only revealed at the moment of block propagation.

2.2.1. Ephemeral Ticket Generation

Each elected validator i generates an ephemeral BLS keypair (sk_i, pk_i) for height H. A lottery ticket T_i is computed as:

$$T_i = \text{Hash}(sk_i \| pk_i \| H)$$

To prove key ownership without exposing the private key, the validator signs the previous block hash ($Hash_{\text{prev}}$) using the ephemeral key:

$$\sigma_i = \text{Sign}_{sk_i}(Hash_{\text{prev}})$$

The validator broadcasts its ticket message (pk_i, σ_i, T_i) to the active **Long-Living Masternode Quorum (LLMQ)**. Each LLMQ consists of exactly 5 members. Once Model D is active, the

quorum signature verification threshold in `CQuorumSignature::Verify()` (`src/llmq.cpp`) is a strict BFT majority (`quorum.members.size() / 2 + 1`), which requires at least **3 out of 5** signatures across all networks (Mainnet, Testnet, and Regtest); prior to Model D activation on Mainnet and Testnet, quorum signature verification is bypassed (0 signatures required) to allow bootstrapping. Furthermore, no hardcoded key ID filtering or local wallet restrictions are applied on Mainnet, Testnet, or Regtest, ensuring a fully decentralized and trustless test environment.

2.2.2. Winner Selection via XOR Distance

The target hash T_{target} is derived from the active rolling seed (Seed_H). The LLMQ calculates the XOR distance between each submitted ticket and the target:

$$D_i = ¿\, T_i \oplus T_{\text{target}} \,¿¿$$

The validator with the smallest distance (D_i) wins the right to propose the block. Ephemeral BLS keys must be derived deterministically from the node's long-term identity key to prevent grinding:

$$sk_i = \text{DeriveKey}\left(sk_{\text{node}}, Hash_{\text{prev}}\right)$$

This ensures that each elected validator can generate exactly one valid ticket per height, preventing ticket pre-computation. DKG sessions to elect new active quorums run every 100 blocks on Mainnet (until block 1,400) and every 10 blocks on Testnet (until block 1,000) / Regtest (until block 800), and rotate every block (`nInterval = 1`) thereafter.

2.2.3. Metric Space and XOR Distance Analysis

The XOR operation $\oplus$ defines a metric space (X, d) on the set of binary keys of length $L = 256$, where $d(x, y) = x \oplus y$. This metric satisfies the three basic properties of a metric space: 1. **Identity of Indiscernibles**: $d(x, y) = 0 \Leftrightarrow x \oplus y = 0 \Leftrightarrow x = y$ 2. **Symmetry**: $d(x, y) = x \oplus y = y \oplus x = d(y, x)$ 3. **Triangle Inequality**: $d(x, z) \leq d(x, y) \oplus d(y, z)$ which in XOR space satisfies the stronger ultrametric property:

$$d(x, z) \leq \max\left(d(x, y), d(y, z)\right)$$

Because T_{target} is pseudorandom and uniformly distributed, and the ephemeral tickets T_i are generated cryptographically, the distance metrics D_i behave as independent, uniformly distributed random variables in $[0, 2^{256} - 1]$. The probability P of any validator winning the block proposal behaves as $1/N$, ensuring complete fairness.

2.2.4. Production Implementation (`src/adam.cpp`, `src/llmq.cpp`, `src/crypto/bls.cpp`)

The production C++ consensus engine unifies the conceptual PoBLS + ADAM mathematical model directly with on-chain deterministic execution: * **Deterministic & Pinned BLS12-381 Key Derivation (`DeriveBLSFromCKey` / `PinBLSPubKey`)**: Every validator, coordinator, and quorum member derives a deterministic BLS12-381 keypair $sk_i = \text{BLST_KeyGen}(\text{SHA256D}(sk_{\text{node}}))$, $pk_i = g_1^{sk_i}$, bound to its permanent `secp256k1` identity key via `CBLSSignedData` (`SignBLSWithECDSAFallback` / `VerifyBLSWithECDSAFallback`). Upon first valid on-chain participation, the `(secp256k1 pubKey -> blsPubKey)` binding is persistently pinned in LevelDB (`PinBLSPubKey`), making key-grinding and intra-block VRF/ticket manipulation mathematically impossible. * **PoBLS Ticket (T_i), Target (T_{target}), and XOR Metric Distance ($D_i = ¿ T_i \oplus T_{\text{target}} ¿¿$)**: Each elected validator's signed partial solution embeds (pk_i, σ_i), from which `ComputePoBLSTicket` derives $T_i = \text{SHA256D}(pk_i \| \sigma_i \| H)$ (`ExtractPoBLSTicketFromSolution`). Against $T_{\text{target}} = \text{SHA256D}(\text{Seed}_H \| H)$ (`ComputePoBLSTarget`), `ComputePoBLSXORDistance` computes $D_i = ¿ T_i \oplus T_{\text{target}} ¿¿$ on $(\{0,1\}^{256}, \oplus)$, and `RankValidatorsByPoBLSDistance` orders active ticket-holding validators in ascending order of XOR distance D_i. * **5-Member BFT LLMQ & BLS12-381 Signature Aggregation (`ElectQuorumMembers`, `CQuorumSignature::Verify`, `CQuorumSignature::AggregateBLSSignatures`)**: For each height H, 5 Masternodes with unspent 4,200 KRISTA collateral (`pcoinsTip->HaveCoin`) and active on-chain status are deterministically elected via $\text{SHA256D}(\text{Seed} \| \text{collateralOutpoint})$. Quorum members sign `blockHash` (`PROPOSEBLOCK` / `QUORUMSIGSHARE`) using deterministic BLS12-381 keys (`DeriveBLSFromCKey`). `CQuorumSignature::Verify` enforces a strict **3-of-5** BFT majority and aggregates the signing members' BLS12-381 public keys (`CBLSPubKey::Aggregate`) and signatures (`CBLSSignature::Aggregate`) to verify the pairing equation $e(g_1, \Sigma_{\text{agg}}) = e(PK_{\text{agg}}, H_2(\text{blockHash}))$. * **Consensus Synergy**: * **Sybil Protection (ADAM)**: Participation is strictly bounded to collateral-backed Masternodes and Coin-Lock / PoW-Lock registered miners. * **DDoS Resistance (PoBLS XOR Ranking + Time-Based Rotation)**: If the primary Coordinator does not propagate a block within 45 seconds (`ADAM_FALLBACK_COORDINATOR_TIMEOUT_SECS = 45`), proposer authority rotates every 30 seconds across the 11 elected validators ranked by smallest PoBLS XOR distance $D_i = ¿ T_i \oplus T_{\text{target}} ¿¿$ (`RankValidatorsByPoBLSDistance`).

2.3. Block Header Extensions & Hashing Chain

When the ADAM upgrade is active, the block header structure is expanded to store consensus proofs:

Field	Type	Description
`vAdamMiners`	`std::vector<CPubKey>`	Public keys of elected validators.
`vAdamSolutions`	`std::vector<std::vector<char>>`	Serialized partial solutions (nonce + signature).
`vAdamVRFProof`	`std::vector<unsigned char>`	Coordinator's VRF signature on the previous seed.
`vAdamCoordinatorSig`	`std::vector<unsigned char>`	Coordinator's signature on the final block hash.

2.3.1. Stateless Hashing Chain

To calculate the block hash, the block header is processed through a sequential hashing chain corresponding to the elected validators. For each validator i in the chain (where $M = \text{size}(vAdamMiners)$):

1. Generate a round-specific hash from the previous block hash and index:

$$\text{roundHash}_i = \text{Hash}(\text{hashPrevBlock} \| i)$$

2. Extract the first byte $v_i = \text{roundHash}_i[0]$ and compute an odd coprime multiplier:

$$m_i = v_i | 1 \ (\text{if } m_i < 3, m_i = 3)$$

3. Execute the hashing round:

- **Fallback Mode (Version 11)**:
  - **Round 0**: $H_0 = \text{CalculateAdamPuzzleHash}(\text{algo}_0, \text{SerializedHeader}) \times m_0 \pmod{2^{256}}$
  - **Round** $i > 0$: $H_i = \text{CalculateAdamPuzzleHash}(\text{algo}_i, H_{i-1}) \times m_i \pmod{2^{256}}$ where the algorithm index is:

$$\text{algoIndex} = \text{Hash}(\text{Seed}_H \| \text{MinerPubKey}_i) \pmod{18}$$

- **Standard Mode (Version 12)**:
  - **Round 0**:
    - Derive 3-permutation algorithms algo1, algo2, algo3 for miner 0.
    - Compute:

$$H_0^{(3)} = \text{CalculateAdamPuzzleHash}(\text{algo3}, \text{SerializedHeader})$$

$$H_0^{(2)} = \text{CalculateAdamPuzzleHash}\left(\text{algo2}, \left(H_0^{(3)} \times 1\right) \pmod{2^{256}}\right)$$

$$H_0 = \text{CalculateAdamPuzzleHash}\left(\text{algo1}, \left(H_0^{(2)} \times 1\right) (\text{mod } 2^{256})\right)$$

- Apply multiplier:

$$H_{\text{prev}} = (H_0 \times m_0) (\text{mod } 2^{256})$$

- **Round** $i > 0$:
  - Derive 3-permutation algorithms algo1, algo2, algo3 for miner i.
  - Compute:

$$H_i^{(3)} = \text{CalculateAdamPuzzleHash}\left(\text{algo3}, H_{\text{prev}}\right)$$

$$H_i^{(2)} = \text{CalculateAdamPuzzleHash}\left(\text{algo2}, \left(H_i^{(3)} \times (i+1)\right) (\text{mod } 2^{256})\right)$$

$$H_i = \text{CalculateAdamPuzzleHash}\left(\text{algo1}, \left(H_i^{(2)} \times (i+1)\right) (\text{mod } 2^{256})\right)$$

  - Apply multiplier:

$$H_{\text{prev}} = (H_i \times m_i) (\text{mod } 2^{256})$$

The final hash H_{prev} (or $H_0 \times m_0$ in Version 11) represents the block hash.

2.3.2. Coprime Multiplier Properties and Mathematical Soundness

The multiplication of the intermediate hashes by m_i modulo 2^{256} is mathematically sound. In modular arithmetic, an element m has a multiplicative inverse modulo K if and only if $\gcd(m, K) = 1$. For the group of integers modulo 2^{256} ($\mathbb{Z}_2^{256}$), the modulus is a power of 2. Therefore, any odd integer m_i is coprime to 2^{256}:

$$\gcd(m_i, 2^{256}) = 1$$

This coprimality guarantees that the mapping $f(x) = x \cdot m_i (\text{mod } 2^{256})$ is a bijection (a one-to-one and onto mapping). As a result: * **No Entropy Loss**: The multiplication preserves the entire entropy of the hash function; no two distinct input values map to the same output value. * **No Degeneracy**: The intermediate state cannot collapse to a zero or sub-space, maintaining the mathematical integrity of the cryptographic chain. * **Non-commutativity**: The ordered application of different multipliers prevents order-swapping attacks.

2.4. Quorum Resiliency and Placeholder Security

A strict cooperative loop requiring 100% participation from all elected validators creates a vulnerability where a single offline node can halt block production. To prevent chain freezes, KristaTech implements a **Quorum-Resilient Responding Mechanism**.

2.4.1. Placeholder Mechanism

The block template is finalized as long as the count of valid validator solutions meets the threshold T governed by `GetAdamThreshold(nHeight)` (5 during bootstrap < `nAdamThresholdBootstrapLimit`, i.e., below block 50,000 on Mainnet / 100,000 on Testnet / 1,000 on Regtest, and 7 on Mainnet/Regtest or 3 on Testnet thereafter). If an elected validator fails to submit its solution within the block slot window, the Coordinator replaces the missing solution in `vAdamSolutions` with an empty byte vector:

$$\text{vAdamSolutions}[i] = \text{std::vector<unsigned char>}()$$

This preserves the positional mapping between `vAdamMiners` and `vAdamSolutions`.

2.4.2. Validation Logic

Validating peers execute the following verification steps in `CheckBlock()`: 1. Verify that `vAdamSolutions` size matches `vAdamMiners` size. 2. Iterate through `vAdamSolutions`. If a solution is empty, it is marked as a placeholder and skipped. If it is non-empty, calculate the puzzle hash and verify the signature and difficulty: * **Version 11**:

```
 $$\text{PuzzleHash} = \text{CalculateAdamPuzzleHash}\left(\text{algoIndex}, \text{Challenge}\right)$$

 where $\text{algoIndex} = \text{Hash}(\text{Seed}_H \mathbin{\Vert} \text{MinerPubKey}_i) \pmod{18}$, and $\text{Challenge} = \text{Seed}_H \mathbin{\Vert} \text{MinerPubKey}_i \mathbin{\Vert} \text{Nonce}_i$.
```

- **Version 12**:

$$\text{PuzzleHash} = \text{algo1}\left(\left(\text{minerIdx}+1\right) \times \text{algo2}\left(\left(\text{minerIdx}+1\right) \times \text{algo3}\left(\text{Challenge}\right)\right)\right) \left(\text{mod } 2^{256}\right)$$

  where algo1, algo2, and algo3 are derived using $\text{GetAdam3PermutationAlgos}\left(\text{hashPrevBlock}, \text{MinerPubKey}_i\right)$, and $\text{Challenge} = \text{Seed}_H \| \text{MinerPubKey}_i \| \text{Nonce}_i$.

1. Confirm that the number of cryptographically validated, non-empty solutions is greater than or equal to T.

2.4.3. Security Proofs

- **Bypass Resistance**: Empty placeholders cannot satisfy signature or difficulty checks. The validation code explicitly treats placeholders as failed proofs. The thermodynamic work

requirement is preserved because at least T nodes must submit valid, high-difficulty proofs.

- **Censorship Non-Profitability**: A malicious Coordinator might try to exclude a miner's solution. However, reward distribution is determined by the protocol based on the elected validator list (`SelectAdamNodes`), regardless of whether a validator's solution was replaced by a placeholder. Because the Coordinator cannot redirect payouts, there is **zero financial incentive** to censor miners.

---

3. MESCAL Smart Contract Language

MESCAL (Minimalistically Envisioned Smart Contract Assembling Language) is a JSON-based declarative language designed to define, assemble, and serialize smart contracts.

3.1. Design Philosophy

Designed for simplicity and safety, MESCAL uses a declarative paradigm that compiles directly into stack-based Bitcoin Script (`CScript`) bytecode. 1. **Safety**: Operates on a restricted, deterministic instruction set matching blockchain opcodes, eliminating re-entrancy vulnerabilities. 2. **Gasless Determinism**: Because loop instructions are omitted, contract execution time is linear and predictable, removing the need for complex gas estimation. 3. **Accessibility**: Contracts are defined as structured JSON, making them easy to generate, verify, and parse across client-side applications.

3.2. Grammar and Formal Syntax

A MESCAL smart contract is defined by a structured grammar. Using Backus-Naur Form (BNF), the contract configuration is represented as:

```
<contract_file>      ::= "{" <declaration_list> "," <contract_def> ","
<active_field> "}"
<declaration_list>   ::= <basic_declaration> | <condition_declaration> |
<declaration_list> "," <declaration_list>
<basic_declaration>  ::= "\"basic\":" "{" <basic_definitions> "}"
<basic_definitions>  ::= <basic_entry> | <basic_definitions> ","
<basic_entry>
<basic_entry>        ::= "\"" <identifier> "\":" "{" <role_def> ","
<inputs_def> "}"
<role_def>           ::= "\"role\":" <opcode_string>
<inputs_def>         ::= "\"inputs\":" "[" <input_list> "]"
<input_list>         ::= <input_entry> | <input_list> "," <input_entry>
<input_entry>        ::= "{" "\"type\":" <type_string> "," "\"value\":"
<value_string> "}"

<condition_declaration> ::= "\"condition\":" "{" <condition_definitions> "}"
<condition_definitions> ::= <condition_entry> | <condition_definitions> ","
<condition_entry>
```

```
<condition_entry>       ::= "\"" <identifier> "\":" "{" "\"role\":" "\"if-
condition\"" "," <exprs_def> "," <true_path> "," <false_path> "}"

<contract_def>       ::= "\"contract\":" "{" "\"" <identifier> "\":" "{"
"\"actions\":" "[" <action_list> "]" "}" "}"
<action_list>        ::= <action_entry> | <action_list> "," <action_entry>
<action_entry>       ::= "{" "\"type\":" <type_string> "," "\"name\":"
<value_string> "}"
<active_field>       ::= "\"active_contract\":" "\"" <identifier> "\""
```

3.3. Opcode Mapping and Specifications

The compiler translates JSON structures into binary operations:

- `hash160`: Pushes the input data and computes SHA256 followed by RIPEMD160.
  - *CScript Equivalent*: `<input> OP_HASH160`
- `check-signature-verification`: Pushes a public key and verifies a cryptographic signature.
  - *CScript Equivalent*: `<pubkey> OP_CHECKSIGVERIFY`
- `equalverify-checksig`: Standard P2PKH validation.
  - *CScript Equivalent*: `OP_DUP OP_HASH160 <pubkeyhash> OP_EQUALVERIFY OP_CHECKSIG`
- `multi-signature`: Multi-signature evaluation.
  - *CScript Equivalent*: `<m> <pubkeys...> <n> OP_CHECKMULTISIG`
- `lock-time`: Enforces time or block-height restrictions.
  - *CScript Equivalent*: `<locktime> OP_CHECKLOCKTIMEVERIFY OP_DROP`
- `number`, `add`, `sub`, `bool-and`, `bool-or`, `num-equal`, `greater-than`, `drop`: Stack arithmetic and boolean primitives (`<int64_t>`, `OP_ADD`, `OP_SUB`, `OP_BOOLAND`, `OP_BOOLOR`, `OP_NUMEQUAL`, `OP_GREATERTHAN`, `OP_DROP`).
- `coin-lock-miner` & `pow-miner`: Specialized ADAM miner registration scripts embedding miner identity, optional PoW challenge/nonce, CLTV locktime, and P2PKH spend rules.

3.4. Execution Safety Proof

Let C be a compiled MESCAL contract consisting of a finite sequence of stack instructions $I_1, I_2, \ldots, I_k$. 1. **Loop-Free Execution**: The instruction grammar contains no loop operations (`OP_LOOP`, `OP_WHILE`, or jumps). Therefore, the control flow graph (CFG) is a directed acyclic graph (DAG). 2. **Linear Time Complexity**: The maximum number of instructions executed is strictly bounded by the number of defined operations:

$$E_{\max} = O(k)$$

Where k is the size of the actions array in JSON. 3. **Termination Guarantee**: Because $E_{\max}$ is finite and linear, every MESCAL contract is guaranteed to terminate in a deterministic number of steps, completely preventing infinite-loop attacks. 4. **Gasless Nature**: Since execution is guaranteed to terminate quickly and linear-time bounds can be verified at compilation, the network does not require gas metering.

3.5. Template Case: Dead Man's Switch (Inheritance)

Allows an heir to claim funds after a period of inactivity, while the owner can access them at any time:

- *CScript Equivalent*:

```
<heir-pubkey> OP_CHECKSIGVERIFY OP_IF <expiry>
OP_CHECKLOCKTIMEVERIFY OP_DROP OP_ELSE <owner-pubkey>
OP_CHECKSIGVERIFY OP_ENDIF
```

```
{
  "basic": {
    "Heir-Sig": {
      "role": "check-signature-verification",
      "inputs": [{ "name": "Pubkey", "type": "pubkey", "value": "02ee1fb8..."
}]
    },
    "Lock-Time": {
      "role": "lock-time",
      "inputs": [{ "name": "Lock-Until", "type": "height", "value":
1780718400 }]
    },
    "Owner-Sig": {
      "role": "check-signature-verification",
      "inputs": [{ "name": "Pubkey", "type": "pubkey", "value": "03ee1fb8..."
}]
    }
  },
  "condition": {
    "Inheritance-Condition": {
      "role": "if-condition",
      "expressions": [{ "type": "basic", "name": "Heir-Sig" }],
      "true": [{ "type": "basic", "name": "Lock-Time" }],
      "false": [{ "type": "basic", "name": "Owner-Sig" }]
    }
  },
  "contract": {
    "Inheritance-Switch": {
      "actions": [{ "type": "condition", "name": "Inheritance-Condition" }]
    }
  },
  "active_contract": "Inheritance-Switch"
}
```

3.6. Taproot (P2TR) Script-Path Integration

MESCAL contracts can be compiled directly into Taproot (P2TR) script-path spending conditions using the built-in `compilemescaltotaproot` RPC command. This allows developers to commit a contract to a Bech32m-encoded P2TR address, preserving privacy and space. * **Compilation**: The compilation process takes the contract JSON and an `internal_pubkey` to construct a script tree. It generates the P2TR `address`, `scriptPubKey`, `leafScript` hex, and a 33-byte `controlBlock`. * **Execution**: To spend the output, a transaction witness script must contain the execution arguments, the `leafScript`, and the `controlBlock` (appended in scriptSig). Since only the executed script leaf is revealed on-chain, alternative branches (e.g., recovery or timeout paths) remain hidden.

---

4. Ecosystem Tokenomics

KristaTech implements a deflationary emission model designed to preserve scarcity while maintaining security incentives over a long lifecycle.

4.1. Core Emission Parameters

- **Maximum Supply (Hard Cap)**: **210,000,000 KRISTA**
- **Premine**: **0 KRISTA** (fair-launch distribution)
- **Block Time Target**: 30 seconds (~1,051,200 blocks per year)
- **Collateral Requirement**: Flat **4,200 KRISTA** from block 1
- **Bootstrap Phase (Blocks 2 - 9,999)**: **100 KRISTA** per block. This phase generates **999,800 KRISTA** (~0.48% of supply), providing enough circulating liquidity to support up to 95 active masternodes before quorum activation.
- **Starting Block Reward (Block 10,000+)**: **15 KRISTA**

4.2. Emission Decay Model

To avoid the supply shocks of 4-year halvings, block rewards decrease gradually using a **1.9% decay every 259,200 blocks**. With a block target spacing of 30 seconds, this 259,200-block interval translates to approximately every 90 days (quarterly) in calendar time. The reward at period P is calculated as:

$$\text{Reward}(P) = 15.0 \times (0.981)^{P}$$

Where:

$$P = \left\lfloor \frac{\text{Height} - 10000}{259200} \right\rfloor$$

4.2.1. Mathematical Derivation of Supply Cap and Gap Reserve

To prove that the emission model never exceeds the hard cap of 210M KRISTA, we represent the total supply as the sum of bootstrap emission and the infinite geometric series of decaying periods. Let S_{max} be the maximum circulating supply.

$$S_{\text{max}} = S_{\text{bootstrap}} + \sum_{P=0}^{\infty} \left(B_{\text{blocks}} \times R_0 \times (1-d)^P \right)$$

Where: * $S_{\text{bootstrap}} = 999,800\,\text{KRISTA}$ (emission from blocks 2 to 9,999) * $B_{\text{blocks}} = 259,200$ (blocks per decay period) * $R_0 = 15.0\,\text{KRISTA}$ (starting decaying reward) * $d = 0.019$ (decay rate of 1.9%, so the multiplier is $1-d = 0.981$)

Since $0 < (1-d) < 1$, the infinite series converges:

$$\sum_{P=0}^{\infty} (0.981)^P = \frac{1}{1-0.981} = \frac{1}{0.019} \approx 52.631579$$

Substituting these constants:

$$S_{\text{max}} = 999,800 + 259,200 \times 15.0 \times \frac{1}{0.019}$$

$$S_{\text{max}} = 999,800 + 3,888,000 \times 52.631579$$

$$S_{\text{max}} = 999,800 + 204,631,579 \approx 205,631,379\,\text{KRISTA}$$

The difference between the Hard Cap ($210,000,000$ KRISTA) and the maximum supply limit S_{max} represents the **Gap Reserve** (G_{reserve}):

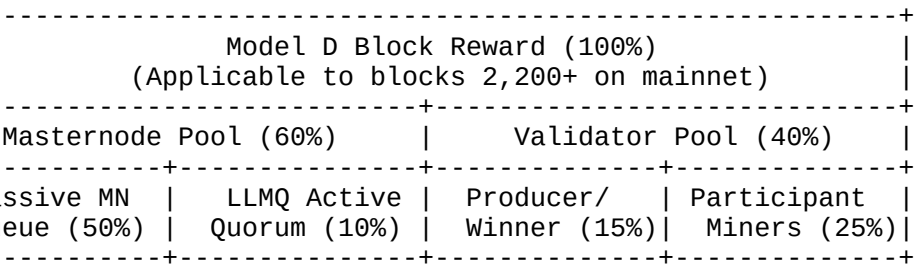
$$G_{\text{reserve}} = 210,000,000 - 205,631,379 = 4,368,621\,\text{KRISTA}$$

This Gap Reserve of **4,368,621 KRISTA (2.08%)** ensures the network can continue reward emissions for decades. This gradual decay prevents security shocks and facilitates a smooth transition to a transaction-fee security model.

```
Supply Saturation Lifecycle:
[0] (Genesis) -> [19.71M] (Year 1) -> [33.45M] (Year 2) -> [68.85M] (Year 5)
-> [112.43M] (Year 10) -> [205.63M] (Asymptote)
```

4.3. Model D Cooperative Reward Split

Block rewards are split to incentivize both validator execution and network infrastructure.



- **Early Stage (Blocks 2 - 2,199)**: 100% Miner-Staker (supports network security while masternodes accumulate).
- **Quorum Accumulation (Blocks 2,000 - 2,199)**: LLMQs activate, prompting nodes to set up masternodes.
- **Model D Activation (Blocks 2,200+)**: The hybrid split is enforced:
  1. **Masternode Passive Share (50%)**: Paid to the next Masternode in the global deterministic queue (incentivizes collateral locking).
  2. **LLMQ Quorum Active Share (10%)**: Distributed among the active masternode members validating and signing PoBLS tickets for that block.
  3. **Block Producer Share (15%)**: Paid to the winning PoBLS validator (PoW block) or coin staker (PoS block), plus transaction fees.
  4. **Validator Participant Share (25%)**: Distributed among the remaining elected validators (divided by 10 for PoS, 11 for PoW), compensating them for local puzzle solving and block validation.

4.3.1. Hybrid Block Reward Dynamics (PoW vs. PoS)

The split adapts dynamically to the block type to support miners and stakers: * **PoW Blocks**: The 40% validator share is distributed among the ADAM validator pool (15% to the coordinator/proposer, 25% split among the 11 miners). * **PoS Blocks**: The 15% producer share goes to the coin staker who won the kernel difficulty check. The 25% validator share is split among the 10 elected ADAM miners, keeping the mining infrastructure active.

4.4. Developer & Faucet Treasury

To fund development and user onboarding, the protocol implements direct treasury allocations: * **Developer Treasury**: **7.0%** of the block reward is allocated to the Developer Fund Address (`KTeRC2Kj8hXcuzRohxPp2LtYFRJm2BceRoP`) from block 2 onward. * **Bootstrap Faucet**: **0.7%** of the block reward is allocated to the Faucet Fund Address (`KThn5Li5BnfGzbNJ92UDH7FL9CZR1VSRGEh`) for blocks 2 through 50,000.

These allocations are deducted directly from the total block value. For example, during the bootstrap phase, the 100 KRISTA block reward is distributed as: 7 KRISTA to the Developer Treasury, 0.7 KRISTA to the Faucet, and the remaining 92.3 KRISTA split according to the active consensus rules.

---

5. Security & Cryptographic Analysis

5.1. Sybil Attacks

An attacker attempting to dominate the validator selection or ticket submission pools faces high economic barriers. Masternode registration requires locking **4,200 KRISTA** per node. To control a majority (e.g., 6 out of 11) of the elected validators in ADAM, an attacker would need to control a significant portion of the active Masternode pool, aligning their financial interests with the security of the network.

5.2. Pre-computation and Nothing-at-Stake

- **Pre-computation**: Because ephemeral BLS keys are derived from long-term node identity keys and the previous block hash, validators cannot pre-compute or grind tickets to skew the PoBLS lottery.
- **Nothing-at-Stake**: In PoS, nodes can sign blocks on multiple forks at no cost. In KristaTech, validating on competing forks requires solving the physical multi-algorithm PoW puzzles for at least T validator seats, imposing a real computational cost that mitigates the nothing-at-stake vulnerability.

5.3. Double Block Signatures

For blocks at heights ¿200 (Cooperative PoS), security is strictly enforced using two cryptographic signatures:

> [!IMPORTANT] **No Single-Signature PoS Blocks:** PoS blocks do not transition the network into a single-signature consensus model. To prevent consensus hijack or verification bypass, every PoS block must carry both signatures. Single-signature blocks (temporary or permanent) are strictly rejected by validating nodes under all circumstances.

1. **ADAM Coordinator Signature (`vAdamCoordinatorSig`)**: Validates that the cooperative validator selection and voting rounds were completed successfully.
2. **Staker Block Signature (`vchBlockSig`)**: Generated using the private key of the staking UTXO, locking the transactions to the block.

Validating nodes require both signatures to be valid, securing the chain against both history-rewriting and consensus-hijacking attacks.

---

6. Conclusion

The KristaTech blockchain protocol presents a cooperative consensus design that addresses key limitations of traditional networks. By separating validator selection (ADAM) from block proposal (PoBLS), the protocol achieves Sybil resistance, proposer anonymity, and defense

against targeted DDoS attacks. The quorum-resilient placeholder mechanism ensures chain liveness, while the MESCAL language provides a secure, gasless, and declarative environment for smart contracts. Supported by a 210 Million emission model with a 1.9% decay rate and the Model D Cooperative Split, KristaTech establishes a balanced incentive structure for miners, stakers, and masternode operators, offering a secure and sustainable framework for decentralized applications.

---

7. References

1. Nakamoto, S. (2008). “Bitcoin: A Peer-to-Peer Electronic Cash System.”
2. Micali, S., Rabin, M., & Vadhan, S. (1999). “Verifiable Random Functions.” *Proceedings of the 40th Annual Symposium on Foundations of Computer Science (FOCS)*.
3. Wood, G. (2014). “Ethereum: A Secure Decentralised Generalised Transaction Ledger.”
4. Boneh, D., Gentry, C., Lynn, B., & Shacham, H. (2003). “Aggregate and Verifiable Signatures from Bilinear Maps.” *Journal of Cryptology.*
5. Maymounkov, P., & Mazieres, D. (2002). “Kademlia: A Peer-to-Peer Information System Based on the XOR Metric.” *International Workshop on Peer-to-Peer Systems.*